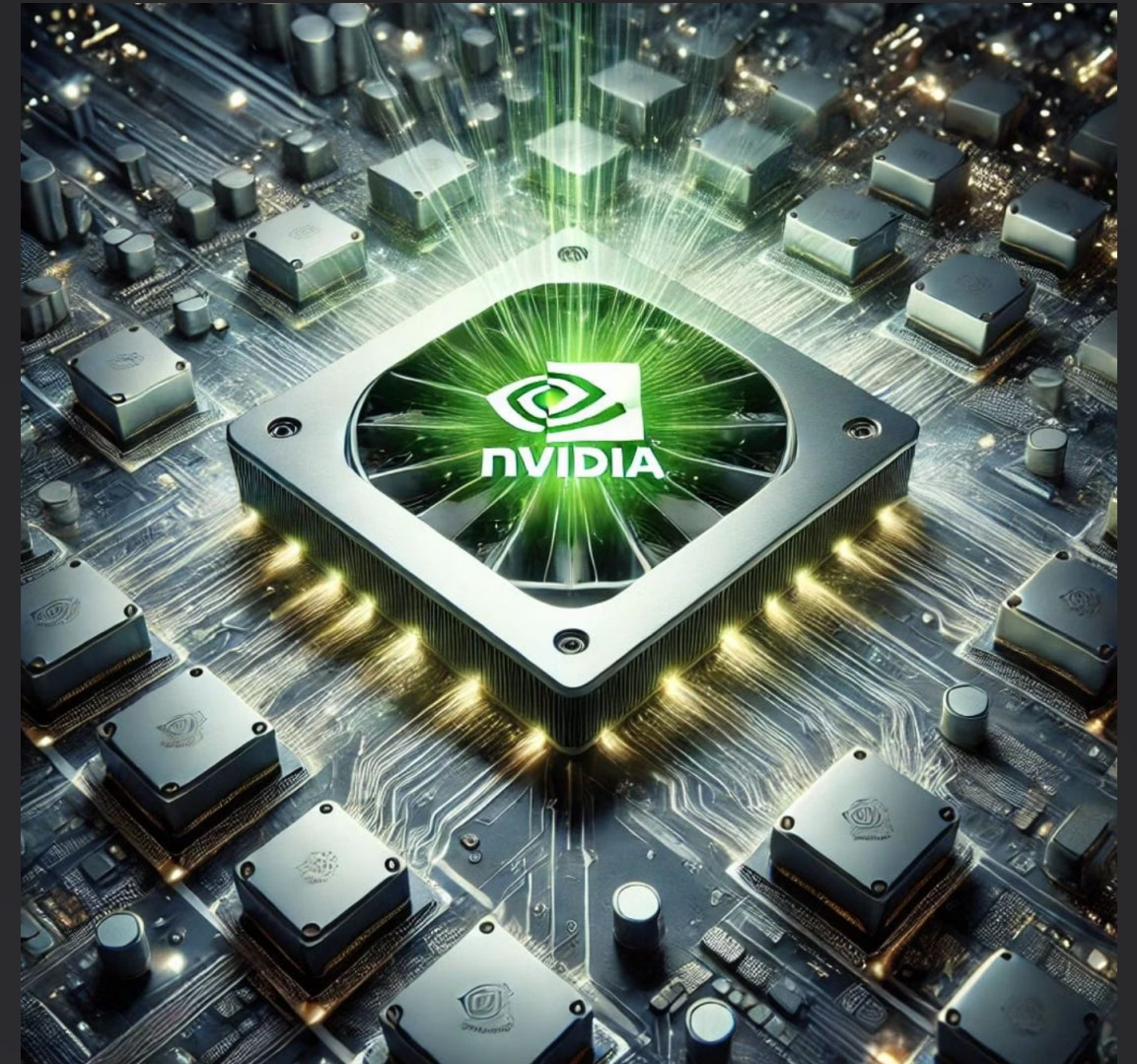


CS131 Fall 2025 Semester Project – Pierce CUDA: Compute Unified Device Architecture

By Harut Payaslyan

Professor Yuen Y.

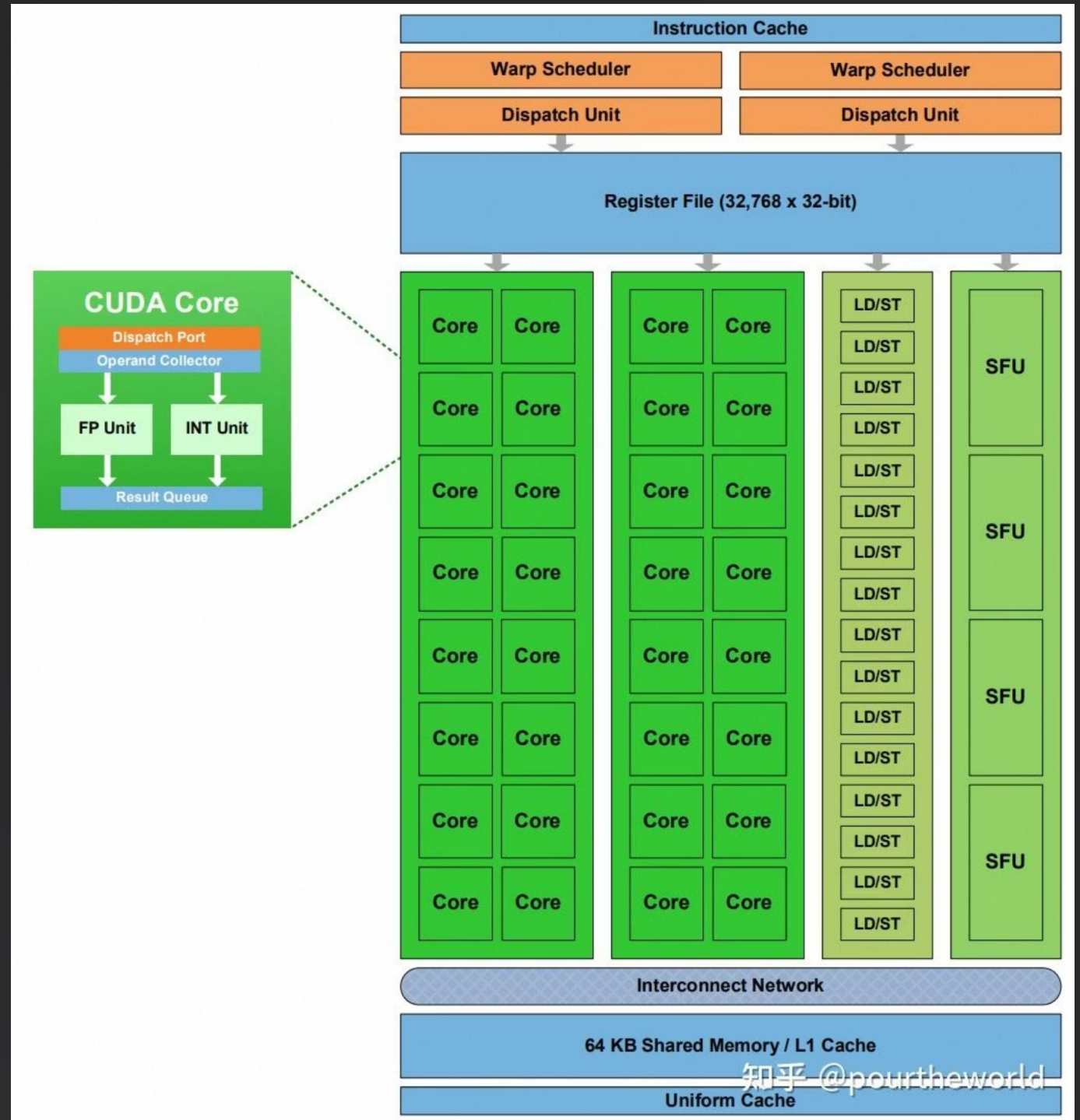


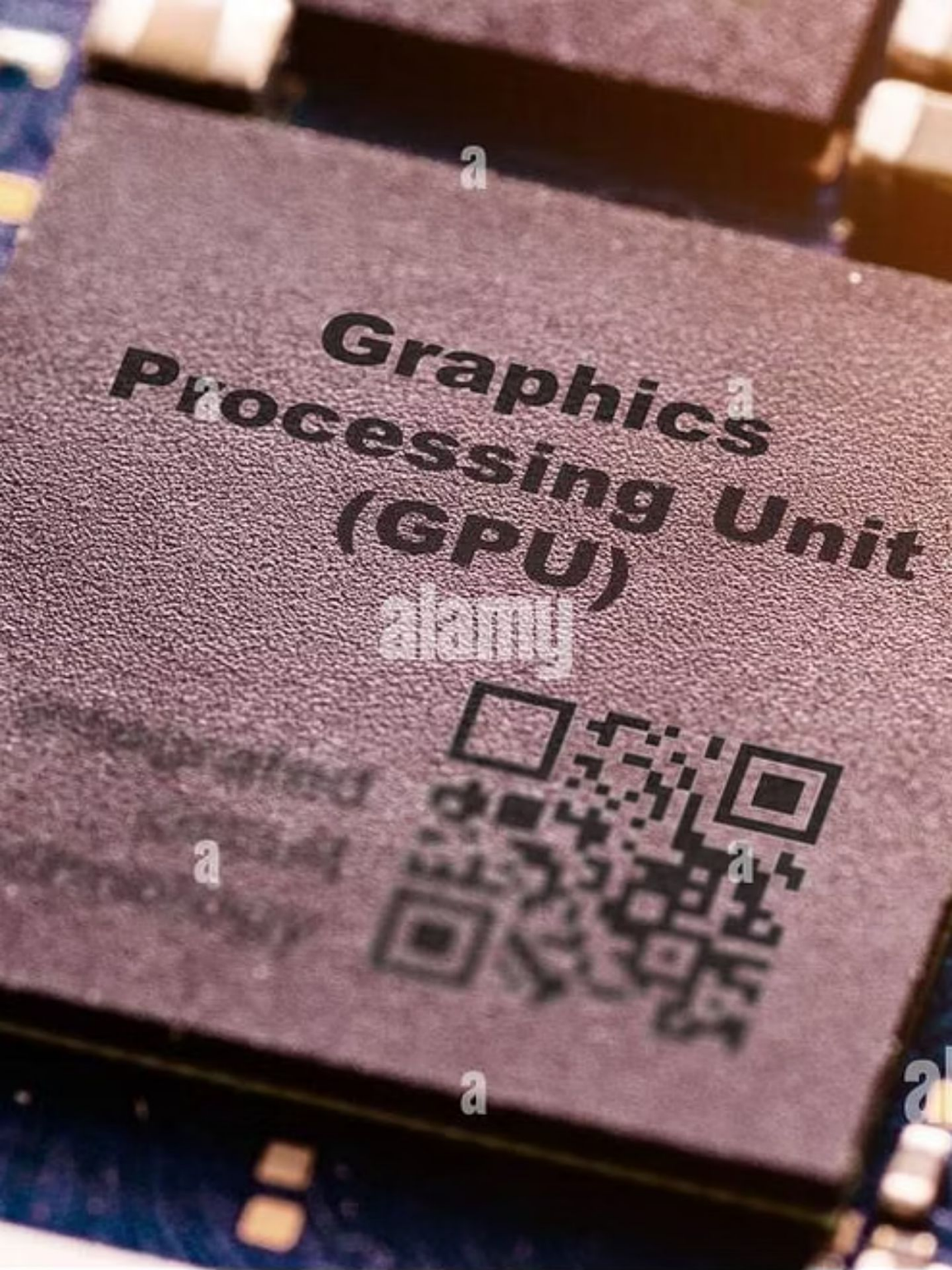
Introduction

What is CUDA?

CUDA is NVIDIA's parallel computing platform and programming model that enables developers to harness the massive parallel processing power of GPUs for general-purpose computing tasks beyond traditional graphics rendering.

Introduced in 2006, CUDA revolutionized high-performance computing by making GPU acceleration accessible through familiar programming languages like C, C++, and Python.





The GPU Architecture Advantage

Thousands of Cores

Modern GPUs contain thousands of smaller, efficient cores designed for parallel execution, compared to CPUs with fewer, more powerful cores.

High Memory Bandwidth

GPUs feature high-bandwidth memory systems capable of transferring hundreds of gigabytes per second, essential for data-intensive applications.

SIMT Architecture

Single Instruction Multiple Thread execution allows thousands of threads to execute the same instruction on different data simultaneously.

CUDA Programming Model

1

Define the Kernel

Write parallel functions (kernels) that execute on the GPU, specifying the computation each thread performs.

2

Allocate GPU Memory

Use `cudaMalloc()` to allocate device memory and prepare data structures for GPU processing.

3

Transfer Data

Copy input data from host (CPU) memory to device (GPU) memory using `cudaMemcpy()`.

4

Launch Kernel

Execute the kernel with specified grid and block dimensions, distributing work across GPU threads.

5

Retrieve Results

Copy computed results back to host memory and free GPU resources with `cudaFree()`.

Thread Hierarchy

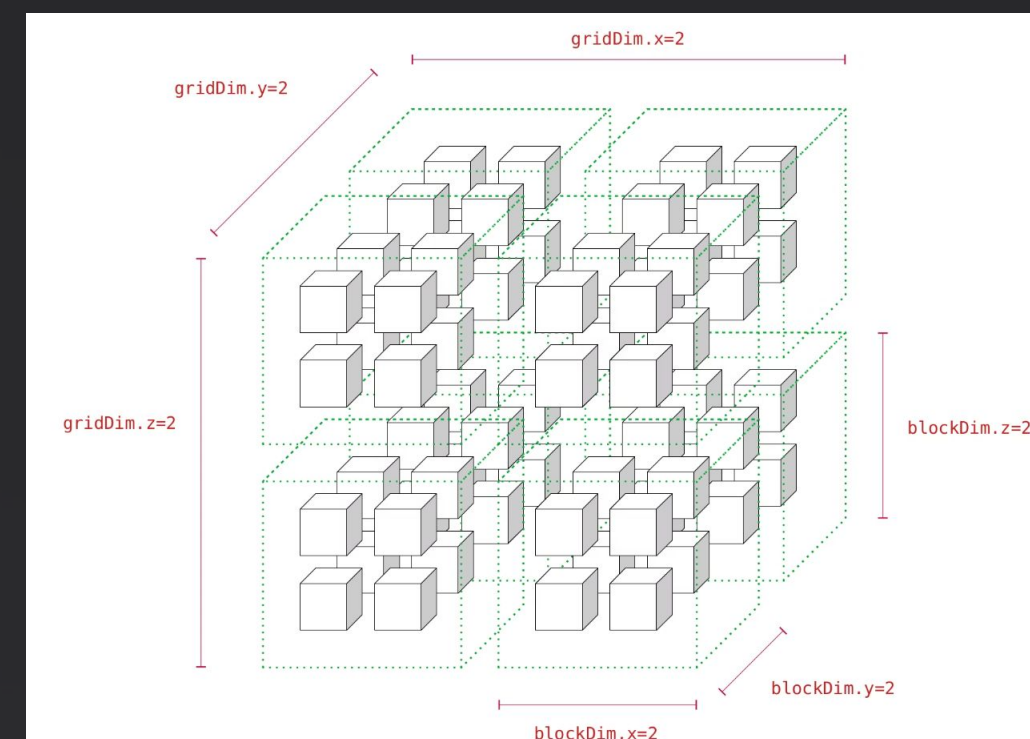
CUDA organizes threads into a hierarchical structure for efficient parallel execution:

Threads: Individual execution units grouped into blocks

Blocks: Collections of threads that share memory and can synchronize

Grids: Arrays of blocks launched together in a single kernel call

This hierarchy maps naturally to GPU hardware, with blocks scheduled on streaming multiprocessors (SMs) and threads executing in groups of 32 called warps.



Memory Hierarchy in CUDA

Global Memory

Large capacity (GBs) but slowest access. Accessible by all threads across all blocks. Persists for the application lifetime.

Shared Memory

Fast, low-latency memory shared among threads within a block. Limited size (~48KB) but crucial for performance optimization.

Registers

Fastest memory, private to each thread. Automatically managed by compiler but limited in quantity per thread.

Constant & Texture Memory

Read-only cached memories optimized for specific access patterns, ideal for lookup tables and spatial data.



Real-World Applications



Medical Imaging

Accelerating CT and MRI reconstruction for 3D visualization in real-time and rapid diagnostic analysis.



Deep Learning

Training neural networks for computer vision, natural language processing, and AI with massive speedups.



Financial Modeling

High frequency trading algorithms, risk analysis, and Monte Carlo simulations to process millions of scenarios.

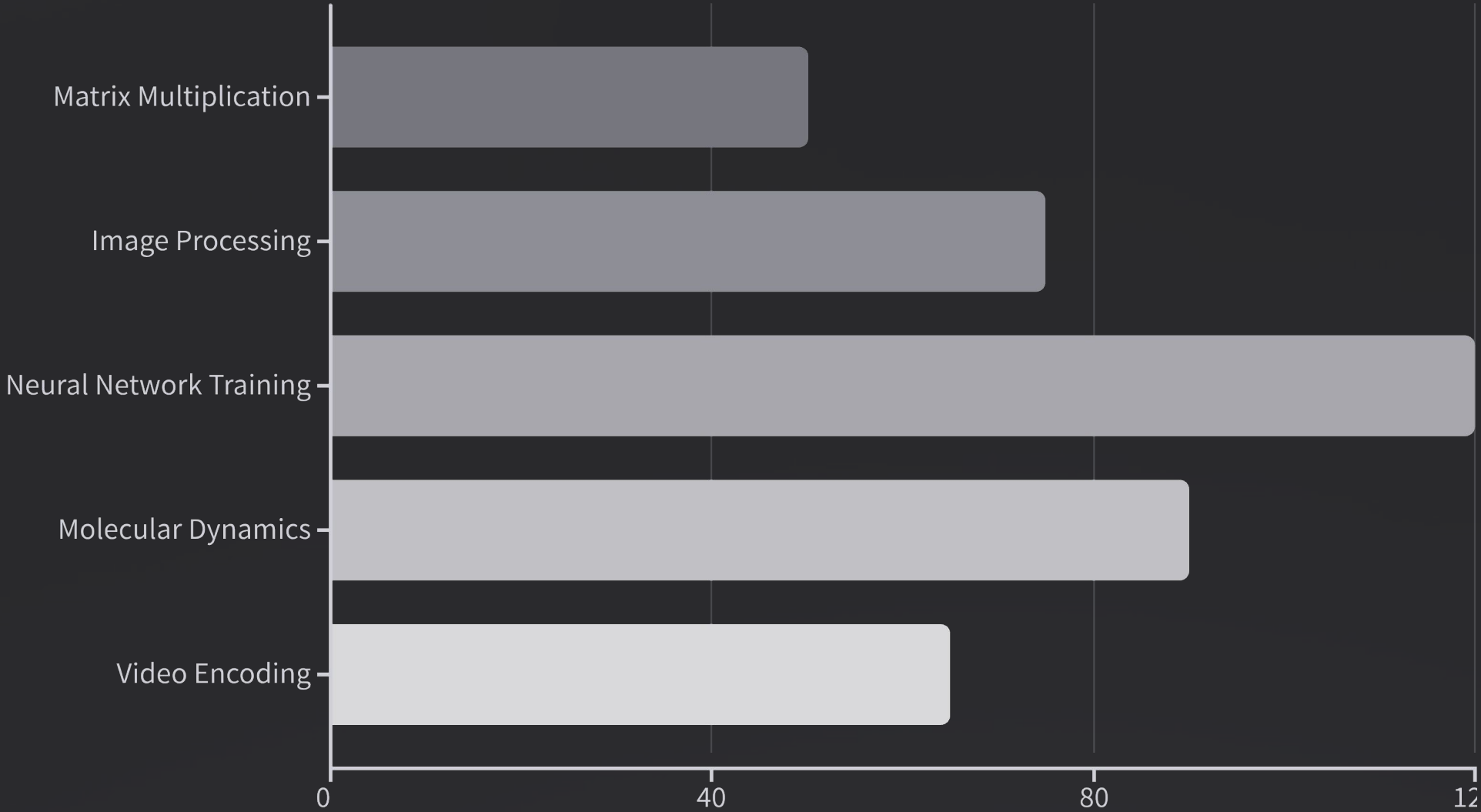


Scientific Computing

Weather simulation, molecular dynamics, computational fluid dynamics, and climate modeling at previously unreachable scales.

Performance Comparison

GPU acceleration delivers dramatic speedups for parallel workloads compared to traditional CPU execution:



These speedups represent typical performance gains when moving from optimized CPU code to well-written CUDA implementations.

Optimization Strategies



Memory Coalescing

Coalesce your global memory accesses, by having threads access sequential addresses



Occupancy Optimization

Tune your block size, register count and shared memory usage to get maximum threads per SM



Minimize Divergence

Minimize warp divergence, threads in a warp must stay in lockstep for highest performance

📌 **CUDA Pro Tip:** Use Nsight tools from NVIDIA to profile your CUDA code, the most significant performance issues can often be related to simple memory access patterns

CUDA Setup and First Steps Guide

Basic Setup

- Install CUDA Toolkit (nvcc compiler)
- Install NVIDIA Nsight for debugging/profiling
- cuBLAS, cuFFT libraries for optimized routines
- Documentation and sample programs are included

Learning Resources

- NVIDIA Developer Zone (tutorials)
- CUDA C Programming Guide
- Online courses (Udacity, Coursera)
- Community forums and GitHub examples



Start with simple examples (vector addition) then progress to more complex apps as you grasp the fundamentals.

Important Lessons



Huge Parallelism

CUDA provides order-of-magnitude speedups for data-parallel tasks by opening up thousands of GPU cores for general-purpose processing.



Programming that is accessible

Developers can learn parallel programming more easily with familiar C/C++ syntax and GPU-specific features.



Optimisation Is Important

Achieving optimal performance requires an understanding of GPU architecture, thread organisation, and memory structures.

By making GPU acceleration feasible and affordable, CUDA has revolutionised scientific computing, artificial intelligence, and numerous other domains. Parallel programming abilities are becoming more and more important as applications become more data-intensive.

The End